UNIVERSITY OF LEEDS

Sequential data

Deep Learning - COMP5203M

2022-23

Sentiment analysis

I was a person that saw all the hype and claims of masterpiece as overreacting and overblown excitement for another Joker based film. I thought this looked solid at best and even a bit too pretentious in the trailer, but in here to say I was incredibly wrong. This is a massive achievement of cinema that's extremely rare in a day and age of cgi nonsense and reboots. While this is somewhat of a reboot of sorts, the standalone origin tale is impeccable from start to finish and echoes resemblance to the best joker origin comics from the past. Joaquin bleeds, sweats, and cries his every drop into this magnificently dedicated performance. Heath Ledger would be proud. This is undoubtedly the greatest acting performance since Heath's joker. The directing and writing is slickly brilliant and the bleak settings and tones are palpable throughout. When this film was over the place was blown away and every audience member was awestruck that they witnessed a film that could still transport them into a character's world and very existence. Believe the hype. This is going to be revered as a transcending masterpiece of cinema.

From IMDb review, 2019

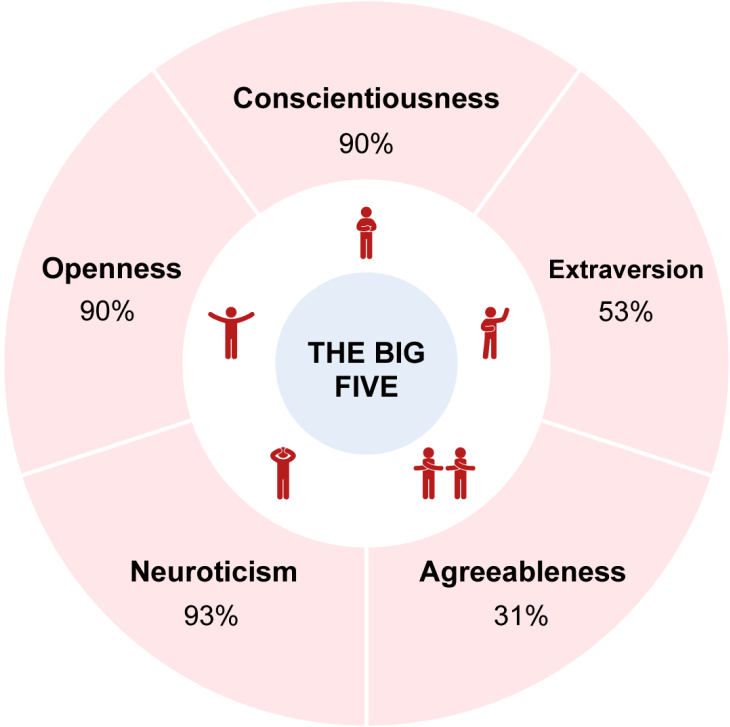
Sequence of characters/words

positive/negative

Personality insights from text

Vector of five personality scores

Well, thank you very much, Jim, for this opportunity. I want to thank Governor Romney and the University of Denver for your hospitality. There are a lot of points I want to make tonight, but the most important one is that 20 years ago I became the luckiest man on Earth because Michelle Obama agreed to marry me. And so I just want to wish …



From IBM Personality Insights service

Image captioning

A man riding a wave on top of a surfboard

sequence of characters/words

Language translation

A woman is running with her dog down an isolated country road → *Une femme court avec son chien sur une route de campagne isolée*

Input and output are both sequences: referred to as *seq2seq*

Text to speech

The next train to Leeds

sequence of audio samples

Text generation

Starter text

Computer generated text

Title: Star's Tux Promise Draws Megyn Kelly's Sarcasm

Subtitle: Joaquin Phoenix pledged to not change for each awards event

Article: A year ago, Joaquin Phoenix made headlines when he appeared on the red carpet at the Golden Globes wearing a tuxedo with a paper bag over his head that read, "I am a shape-shifter. I can't change the world. I can only change myself." It was a promise to not change to fit into the Hollywood mold: "I think that's a really special thing, to not change yourself. I think it's a really special thing to say, `This is what's inside of me, I'm proud of it, and I'm not going to be ashamed because of the way that someone else thinks I should be.'" Now, it's the Oscars, and Phoenix is at it again. But this time, his publicist is saying he'll be wearing a tux no matter what. Megyn Kelly was not impressed, and she let him have it on The Tonight Show. "You know, I feel like, I feel like you could have worn the tux," she says. "But you're saying you're a shape-shifter. I don't know if you can change your tux, but you can change your mind. You can change your mind. You can change your mind." Phoenix says he did, but it didn't stick. "I was like, `Okay, I'm going to wear a tuxedo to this thing.' And then I thought, `I don't want to wear a tuxedo to this thing.'" Kelly goes on to encourage him to change his mind again, but Phoenix says it's too late: "I'm committed to wearing this."

From *Language models are few shot learnings, NeurIPS 2020.*

Stochastic processes

Probability distribution function for the next output given the past

$$p(\boldsymbol{x}_t | \boldsymbol{x}_1, \cdots, \boldsymbol{x}_{t-1})$$

defines a *stochastic process*

Sample from this distribution to generate different possible outputs.
Repeat to generate a sequence of outputs.

Approximate this function by only looking at the past n time steps:

$$p(\boldsymbol{x}_t | \boldsymbol{x}_{t-n}, \cdots, \boldsymbol{x}_{t-1})$$

This is an *n'th order Markov process*

UNIVERSITY OF LEEDS

Recurrent Neural Networks

Deep Learning - COMP5203M

2022-23

Representing text as sequences

We represent text strings as sequences of *tokens* from a *vocabulary.*

For the text string ‘The cats ran quickly’

The tokens are individual words

'the' 'cats' 'ran' 'quickly’

The tokens are individual characters

't' 'h' 'e' <space> 'c' 'a' 't' 's' <space> 'r' 'a' 'n' <space> 'q' 'u' 'i' 'c' 'k' 'l' 'y '

'the' <space> 'cat' 's' <space> 'ran' <space> 'quick' 'ly'

The tokens are characters, partial words, and words.

One-hot encoding of tokens

Consider a vocabulary of N tokens. Assume the vocabulary is ordered so that each token has a index.

In the *one-hot* vector encoding, we represent a token by a vector of zeros, with a one in the position of the token index:

$$\underbrace{[0\ 0\ 0\ 0\ \ 0\ \cdots 0\ 1\ 0 \cdots 0\ 0\ 0\ 0\ 0]}_{\text{length N}}$$

Not just used for text

Consider the 10 classes of CIFAR-10:

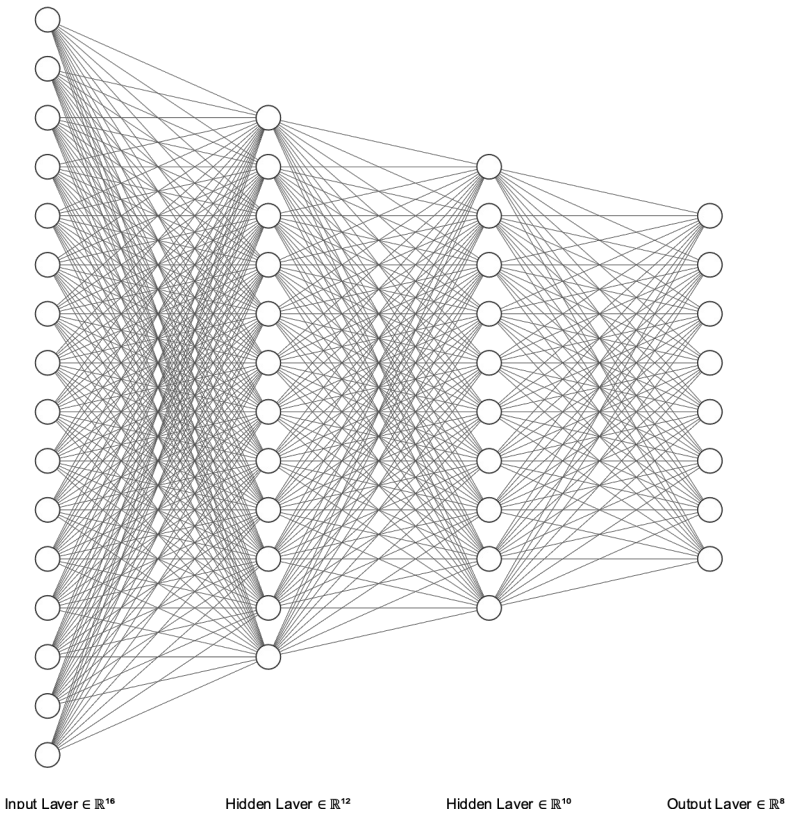
(airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck)

In the *one-hot* vector encoding, we represent the class 'bird' by a binary vector:

$$[0\ 0\ 1\ 0\ 0\ 0\ 0\ 0\ 0\ 0]$$

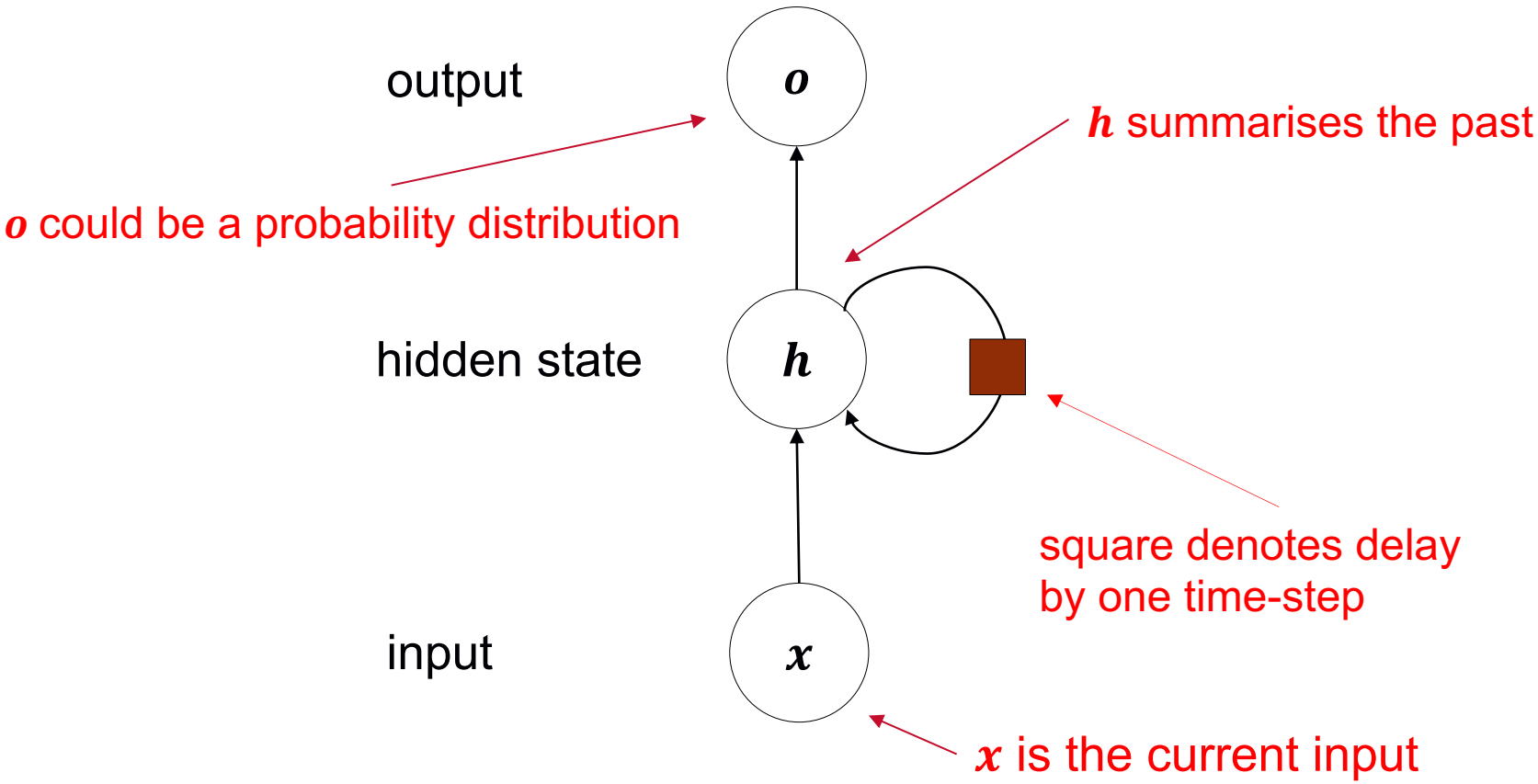
Why recurrent neural networks

- Input and outputs are of variable length
- MLPs do not share features learned across different positions of the input sequence.



Elman Recurrent Neural Network (RNN)

Also known as a *vanilla* RNN

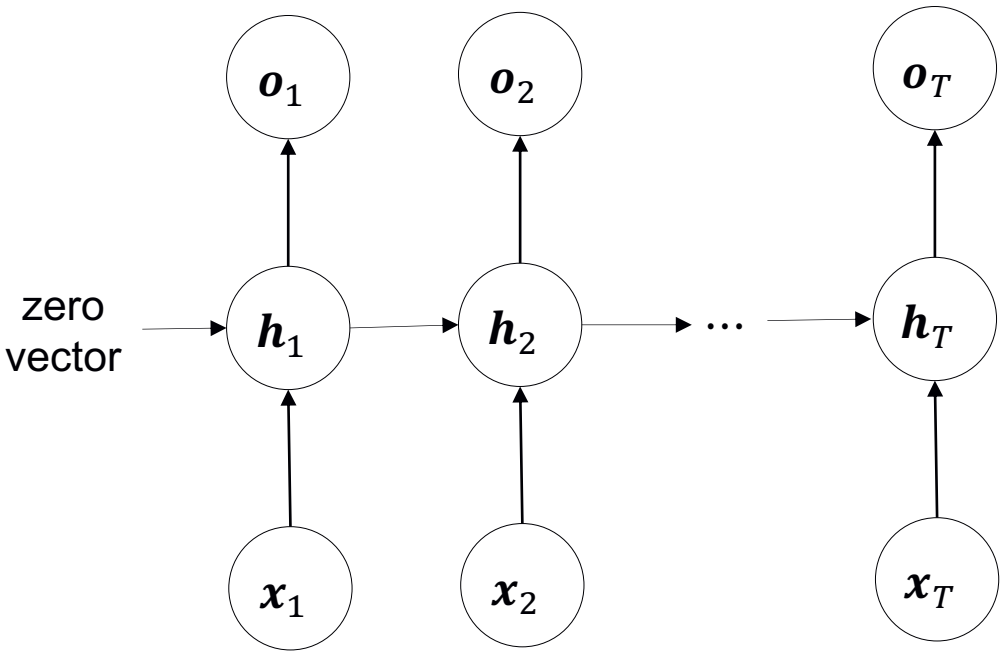


Operates over multiple time-steps.

There is no limit to the number of time-steps.

Operate over T time steps to give a computation graph

unrolling



$$o_t = \mathbf{c} + \boldsymbol{V}\boldsymbol{h}^t$$

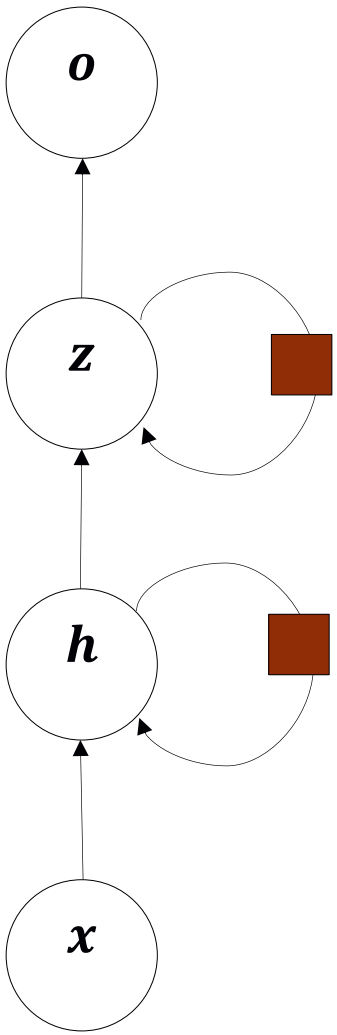
$$\boldsymbol{h}_t = tanh(\boldsymbol{b} + \underbrace{\boldsymbol{W}\boldsymbol{h}_{t-1} + \boldsymbol{U}\boldsymbol{x}_t}_{\text{can be written as } [\mathbf{W},\mathbf{U}][\boldsymbol{h}_{t-1};\boldsymbol{x}_t]})$$

$\boldsymbol{U}, \boldsymbol{V}, \boldsymbol{W}$ are weight matrices

$\boldsymbol{b}, \boldsymbol{c}$ are bias vectors

The same for all time-steps

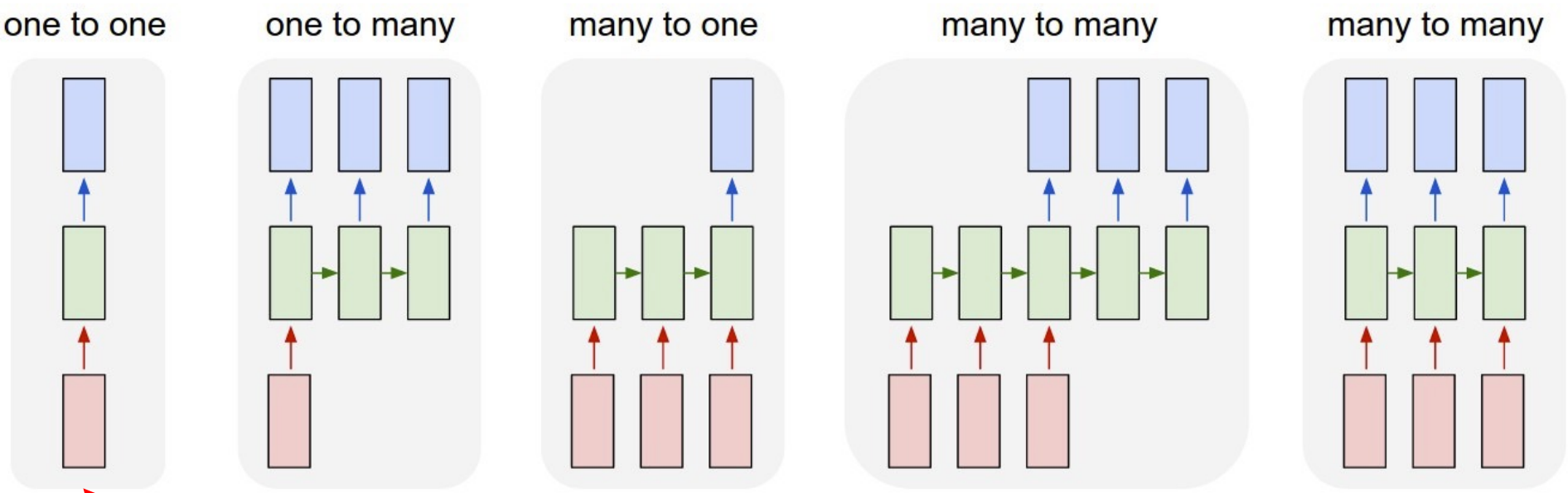
Deep RNNs



Add multiple recurrent hidden state layers

RNNs map from input sequence to output sequence

Several variations of this *seq2seq* mapping



Regular non-recurrent network

Diagram from Andrej Karpathy blog, http://karpathy.github.io/2015/05/21/rnn-effectiveness/, accessed March 2021.

Language Models

- A language model gives you the probability of a given sentence under the corpus of text it has been trained on.
- The probability distribution can be learned using an RNN.
- The resulting model can be used for novel text generation by sampling from the distribution.

$$p(\boldsymbol{y}_1, \ldots, \boldsymbol{y}_T) = p(\boldsymbol{y}_1)p(\boldsymbol{y}_2|\boldsymbol{y}_1) \ldots p(\boldsymbol{y}_T|\boldsymbol{y}_1, \ldots, \boldsymbol{y}_{T-1})$$

Using an RNN for text generation

Aim to extend a given piece of text (in blue), as in:

```
The queen looked at alice and she said to herself
and she said to herself in a mourefully as she
could not and sometimes to say it at all said the
caterpillar. well to her like the while and the
dormouse she said to herself in a mouse them the
mock turtle said and so she said the mock turtle
to say what i should like to the hatter with a
sort of catter that the dormouse was a little
began with a streak the dormo
```

Using an RNN for text-clip generation

Aim is to generate novel ‘text-clips’ such as words, image captions, and sentences, as in:

hello

someone on a surf-board

the cat sat on the mat

Start with a corpus of text-clips and use to train an RNN.

Use the RNN to generate novel text-clips.

Tokenise text-clips by character or word.

Here we use characters:

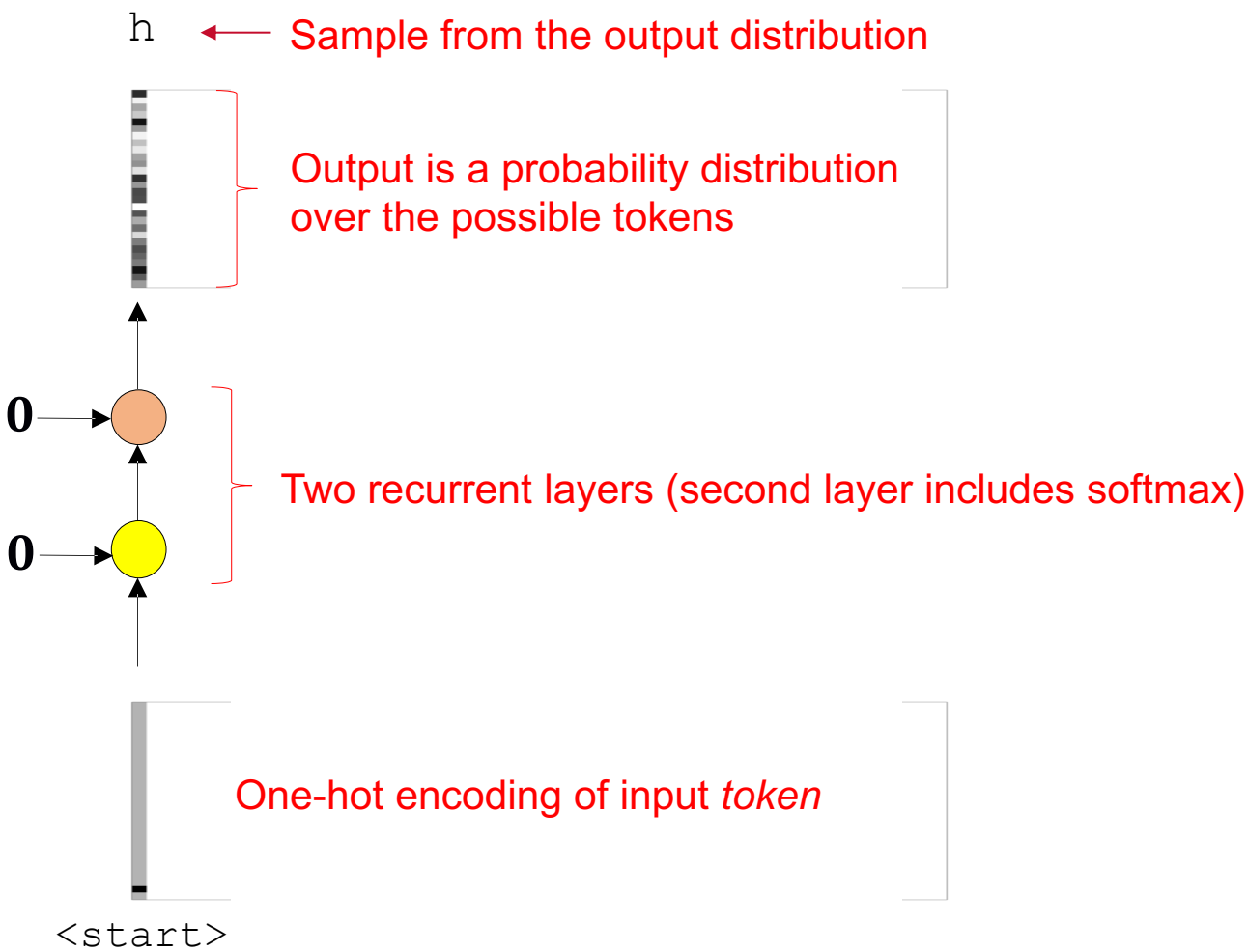
space

29 tokens representing:

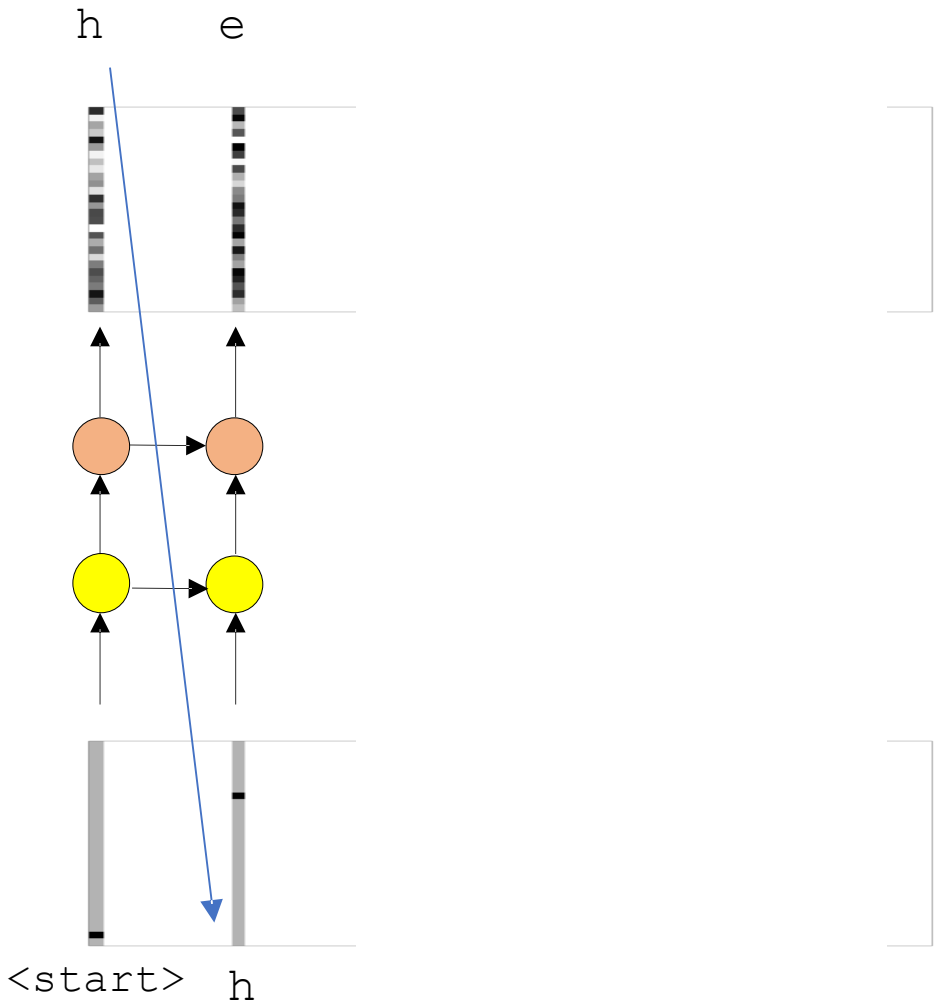
abcdefghijklmnopqrstuvwxyz■<start><end>

Text-clip generation with a two-layer RNN

$t = 1$

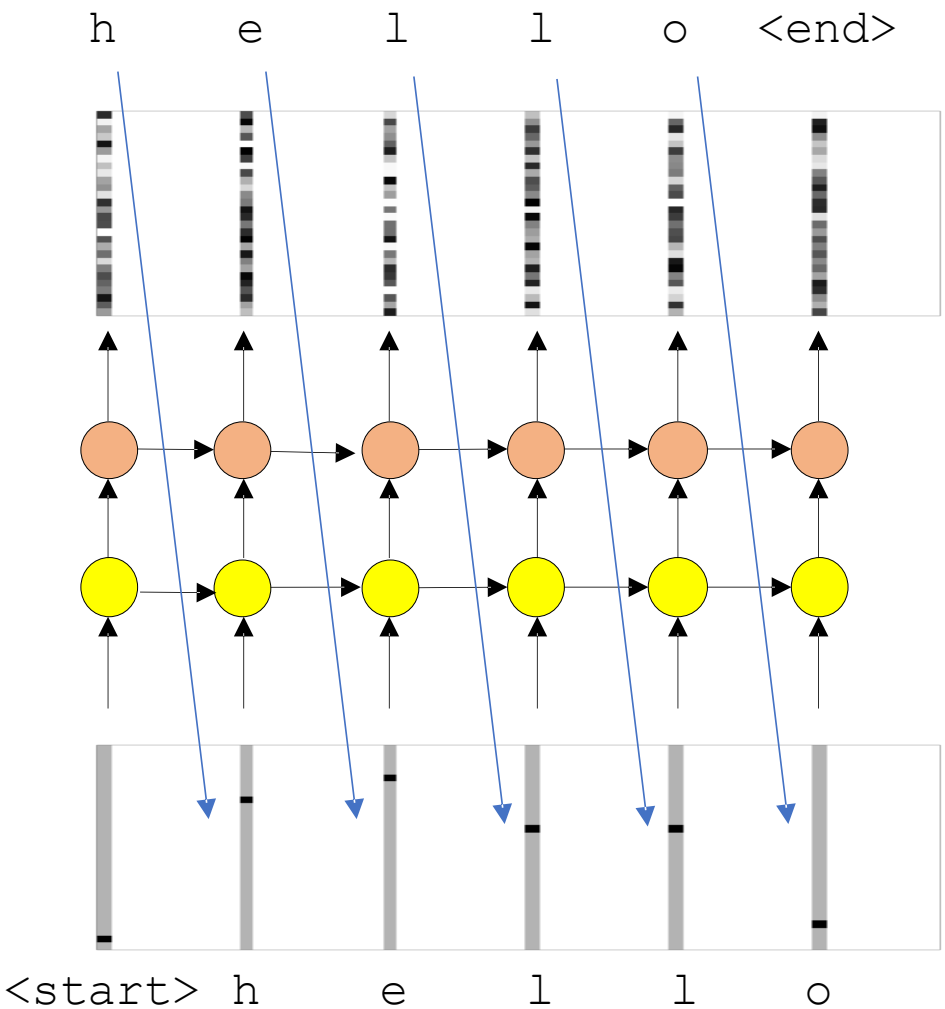


Text-clip generation with a two-layer RNN



$t = 2$

Text-clip generation with a two-layer RNN

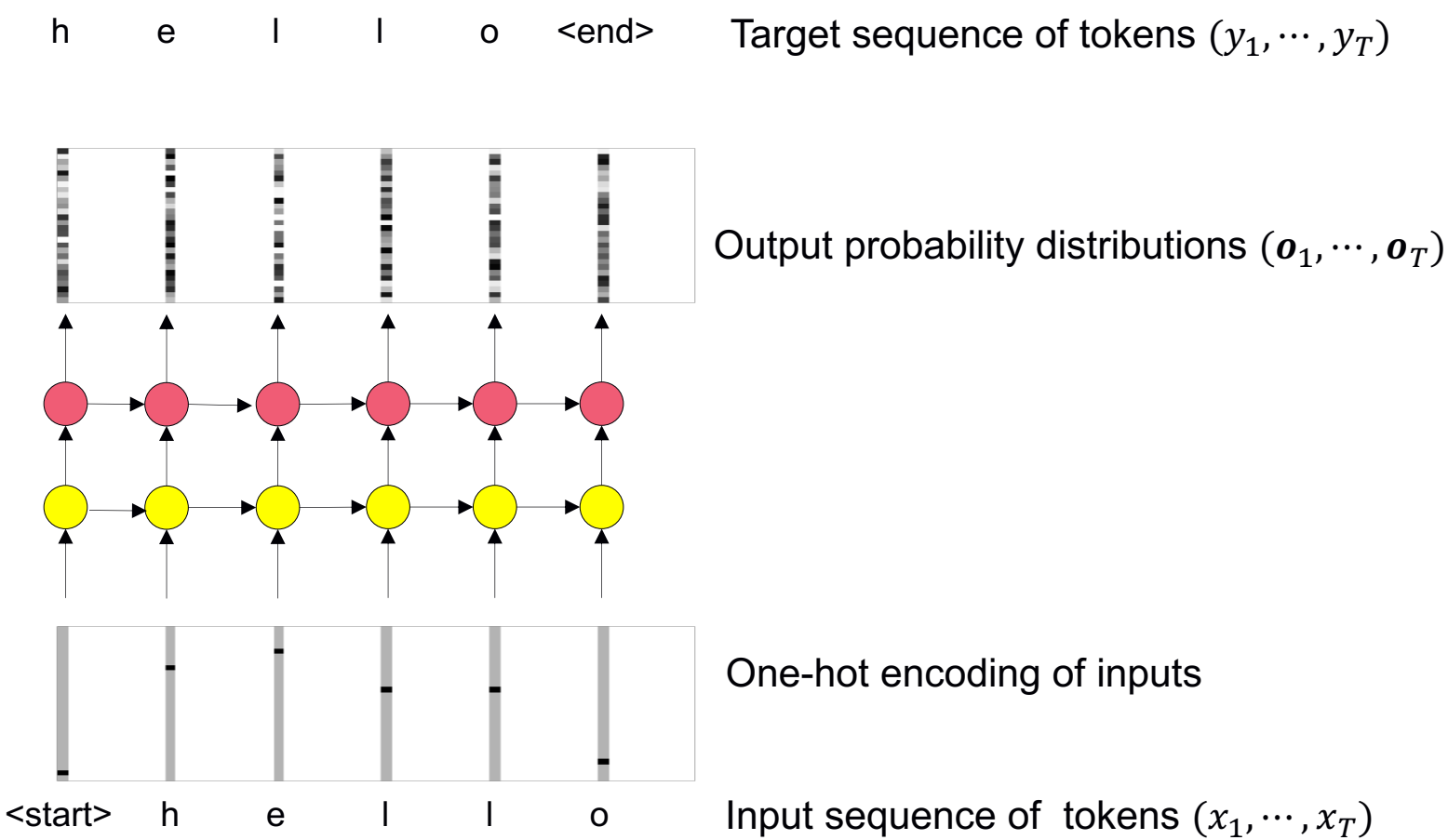


$t = 6$

Stop when the output token is <end>

In some applications, take most probable token at each time step

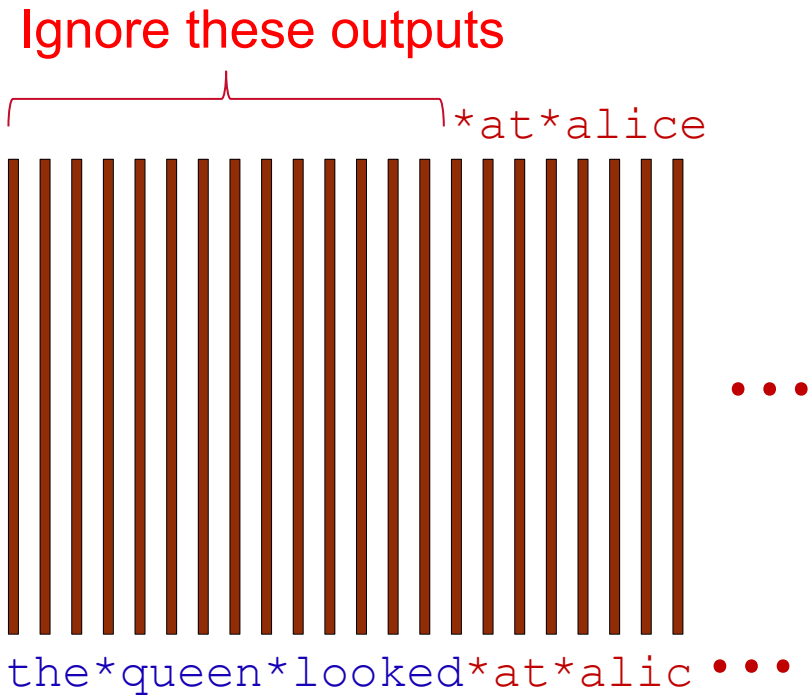
Training the RNN: defining the loss



At each time-step, compute negative log-likelihood of target token y_t given distribution $\boldsymbol{o}_t$. Sum over all T time-steps. Sum over a minibatch of text-clips to give overall loss.

Use backpropagation to reduce the loss on the minibatch.

Using the text generator



Training the text generator

Select a large corpus of text, for example:

- The Complete Works of Shakespeare
- All of the text on the internet

Tokenise the corpus.

Extract a large number of fixed-length token sequences from the corpus (e.g. 64 tokens in length). For example (treating characters as tokens):

`the■cat■sat■on■the■mat`

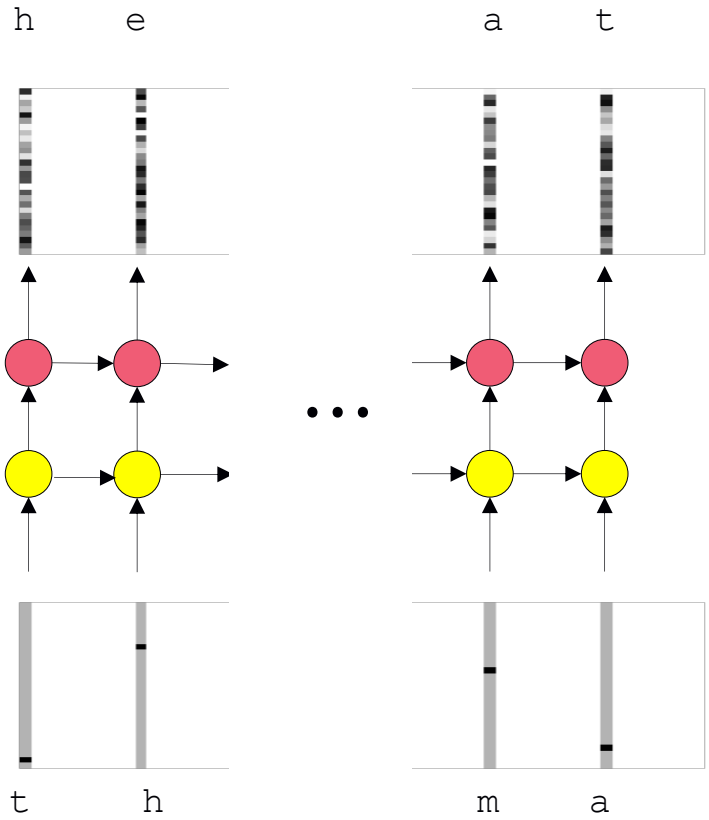
For each example, drop the final token and input to RNN:

`the■cat■sat■on■the■ma`

Compute negative log-likelihood from target (example, omitting first token).

`he■cat■sat■on■the■mat`

Training the text generator



the■cat■sat■on■the■mat

Vanishing gradients

I grew up in France… I speak fluent French

Predicting this word requires information from earlier in the sentence

Although the RNN hidden state can in principle encode information from the distant past, learning to do this doesn't work in practice.

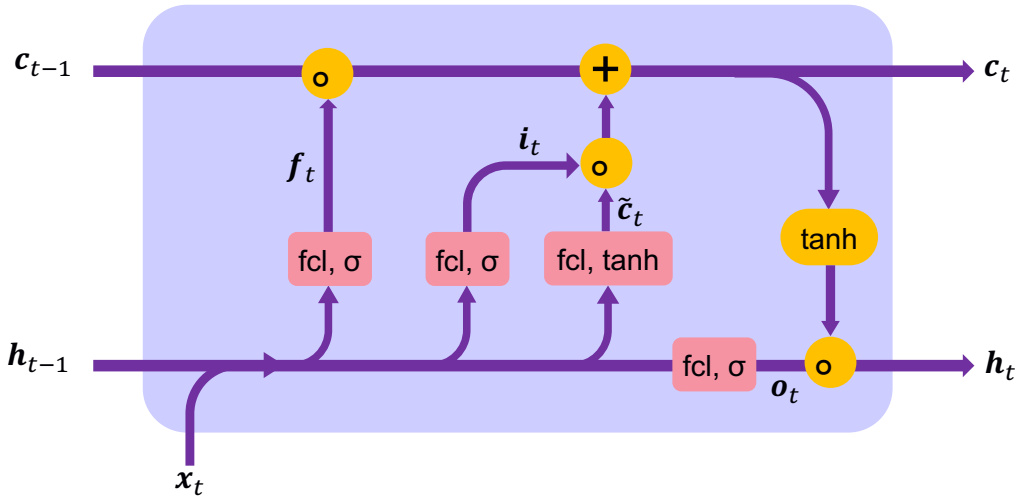
Gradients get vanishingly small the further back in time you go.

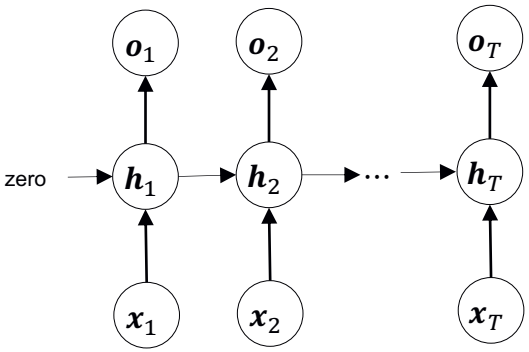
This can obscure essential information for a later prediction.

Popular solution is to use a Long Short-Term Memory (LSTM) or Gated Recurrent Unit (GRU).

Long Short-Term Memory (LSTM)

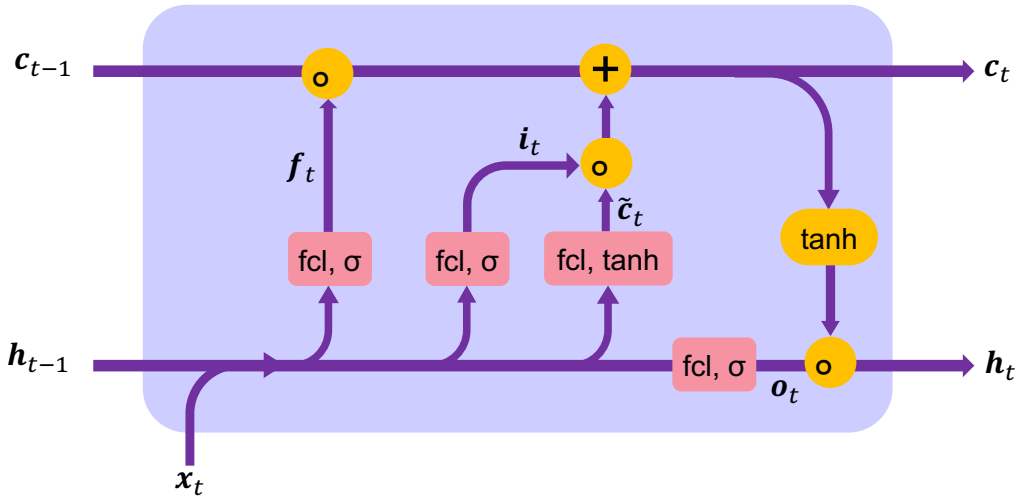
Replace vanilla RNN cell by this:

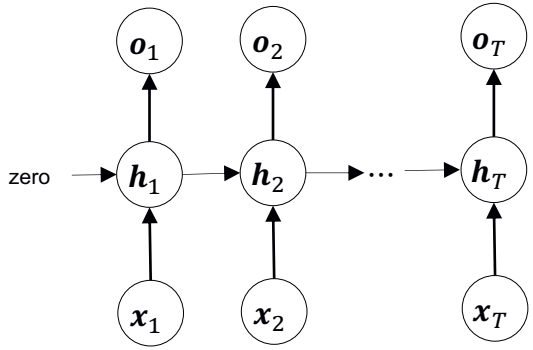




Long Short-Term Memory (LSTM)

Replace vanilla RNN cell by this:





$$\boldsymbol{f}_t = \sigma(\boldsymbol{W}_f \boldsymbol{x}_t + \boldsymbol{U}_f \boldsymbol{h}_{t-1} + \boldsymbol{b}_f)$$

$$\boldsymbol{i}_t = \sigma(\boldsymbol{W}_i \boldsymbol{x}_t + \boldsymbol{U}_f \boldsymbol{h}_{t-1} + \boldsymbol{b}_i)$$

$$\boldsymbol{o}_t = \sigma(\boldsymbol{W}_o \boldsymbol{x}_t + \boldsymbol{U}_o \boldsymbol{h}_{t-1} + \boldsymbol{b}_o)$$

$$\tilde{\boldsymbol{c}}_t = tanh(\boldsymbol{W}_c \boldsymbol{x}_t + \boldsymbol{U}_c \boldsymbol{h}_{t-1} + \boldsymbol{b}_c)$$

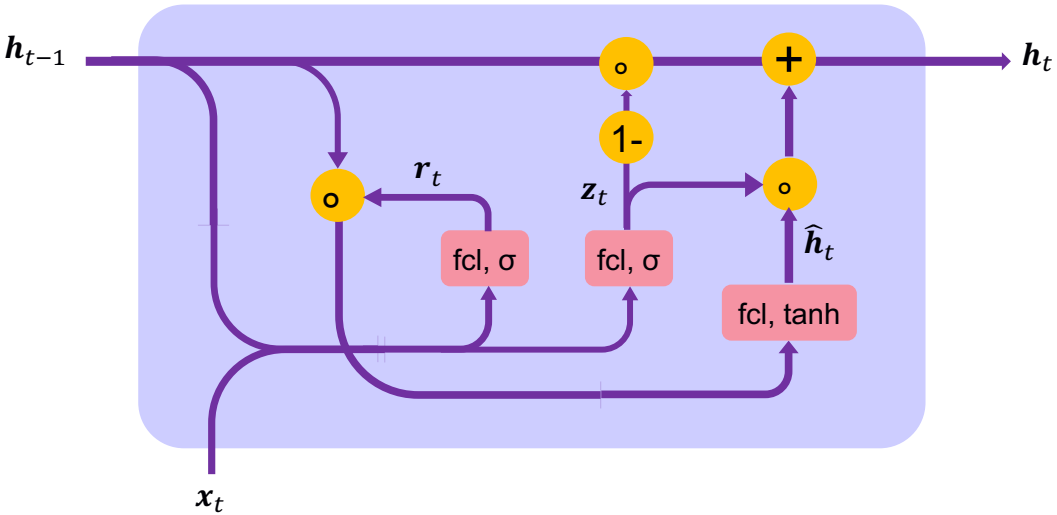
$$\boldsymbol{c}_t = \boldsymbol{f}_t \circ \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \circ \tilde{\boldsymbol{c}}_t$$

$$\boldsymbol{h}_t = \boldsymbol{o}_t \circ tanh(\boldsymbol{c}_t)$$

Hadamard product ≡ element-wise product

$(0,0.5,1.0) \circ (3.0,4.0,2.0) = (0,2.0,2.0)$

Gated Recurrent Unit (GRU)

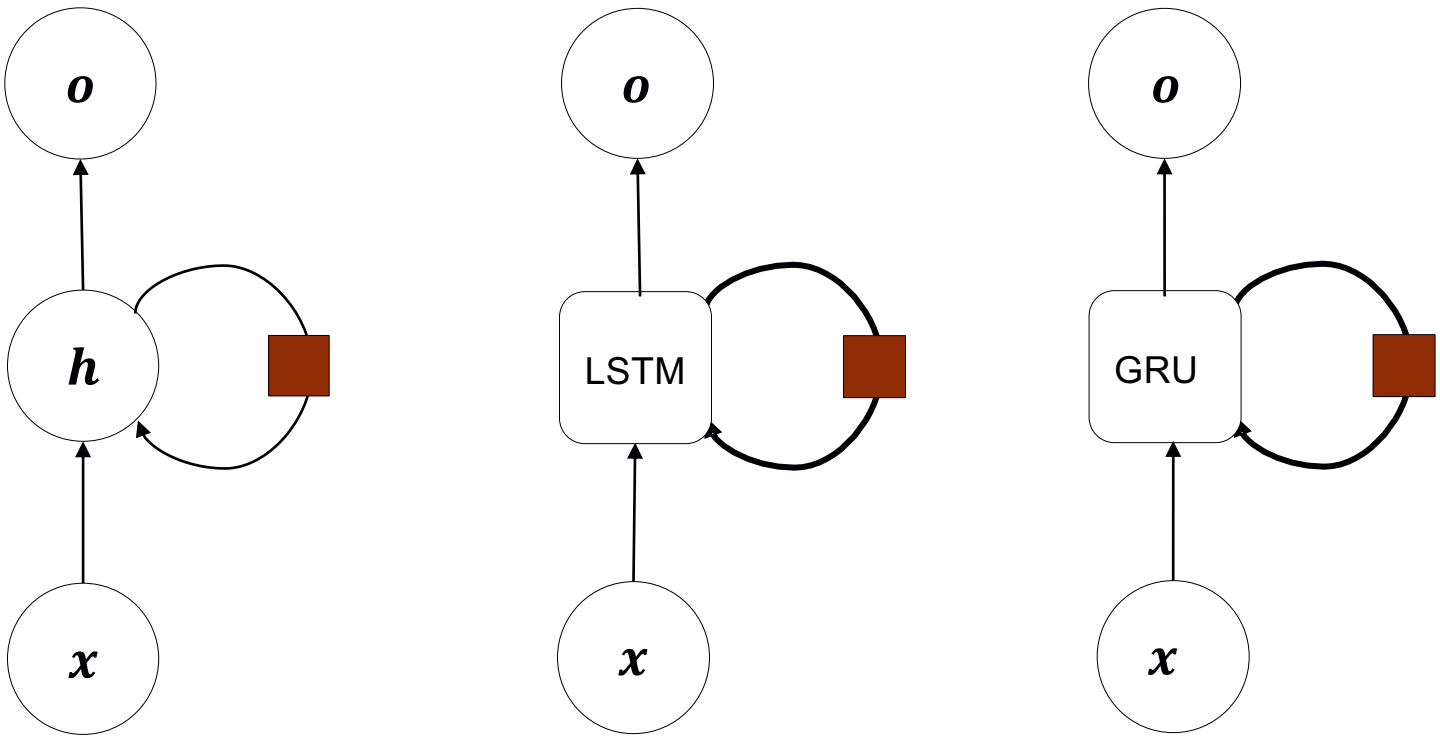


$$\boldsymbol{z}_t = \sigma(\boldsymbol{W}_z \boldsymbol{x}_t + \boldsymbol{U}_z \boldsymbol{h}_{t-1} + \boldsymbol{b}_z)$$

$$\boldsymbol{r}_t = \sigma(\boldsymbol{W}_r \boldsymbol{x}_t + \boldsymbol{U}_r \boldsymbol{h}_{t-1} + \boldsymbol{b}_r)$$

$$\widehat{\boldsymbol{h}}_t = tanh(\boldsymbol{W}_h \boldsymbol{x}_t + \boldsymbol{U}_h(\boldsymbol{r}_t \circ \boldsymbol{h}_{t-1}) + \boldsymbol{b}_h)$$

$$\boldsymbol{h}_t = (1 - \boldsymbol{z}_t) \circ \boldsymbol{h}_{t-1} + \boldsymbol{z}_t \circ \widehat{\boldsymbol{h}}_t$$


o
h
x
o
LSTM
x
o
GRU
x

UNIVERSITY OF LEEDS

Text classification

Deep Learning - COMP5203M

2022-23

A useful encoding for word tokens

In *word embedding*, map each word token into an M dimensional space such that similar words are close together Usually, $M \ll N$

word2vec

Insight:

Words that occur in similar contexts tend to have similar meanings.

This is known as the distributional hypothesis and dates back to the 1950's.

For example, the words oculist and eye-doctor tend to occur in the same context with words like eye and examined.

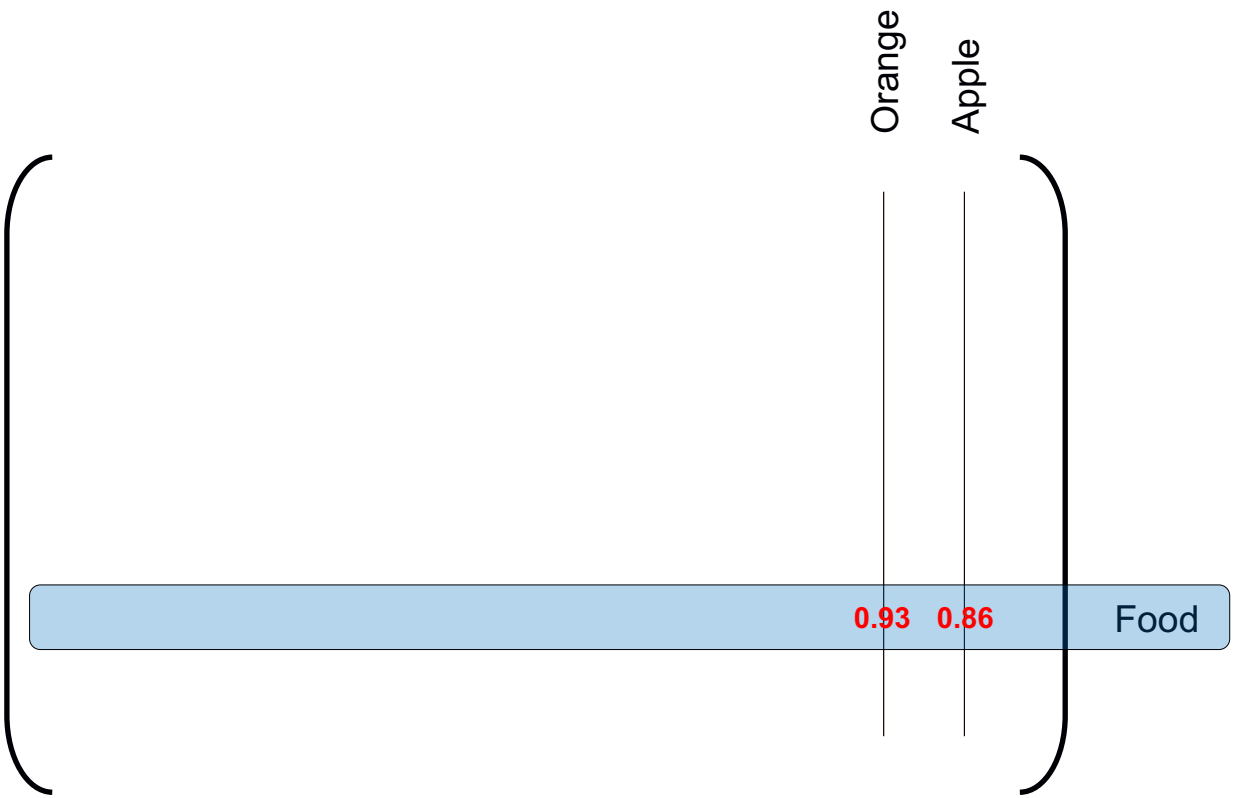
I called the oculist to arrange an eye examination

He became an eye-doctor after examining my eyes

Background in chapter 6 of "Speech and Language Processing" by Dan Jurafsky and James H Martin. https://web.stanford.edu/~jurafsky/slp3/

Word Embedding

$$\mathbf{E}_{d\times|V|} =$$



Vector semantics

Exploit this idea to represent the meaning of words as vectors.

Words with similar contexts are represented by similar vectors.

The word meanings are said to be *embedded* in the vector space.

Suppose you don't know what *ongchoi* means but see these six contexts:

Ongchoi is delicious sauteed with garlic.

Ongchoi is superb over rice.

...ongchoi leaves with salty sauces...

...spinach sauteed with garlic over rice...

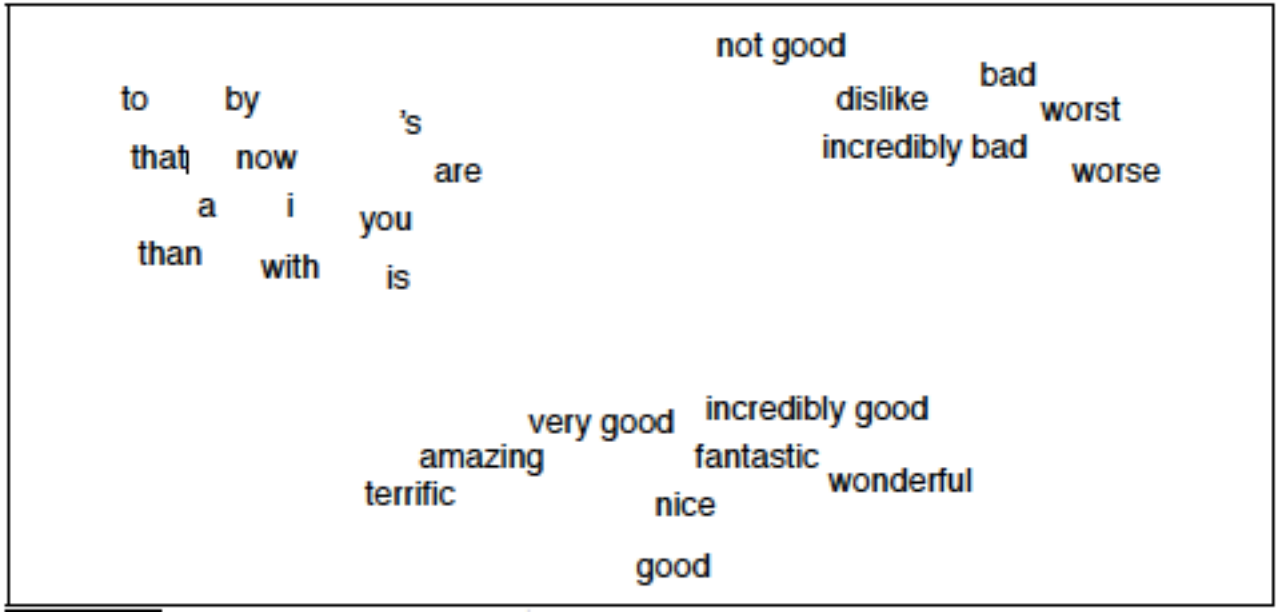
...chard stems and leaves are delicious...

...collard greens and other salty leafy greens

Context words in blue suggest *ongchoi* has similar meaning to words in green

Formalize this by counting the words in context of *ongchoi*.

Example of a word embedding



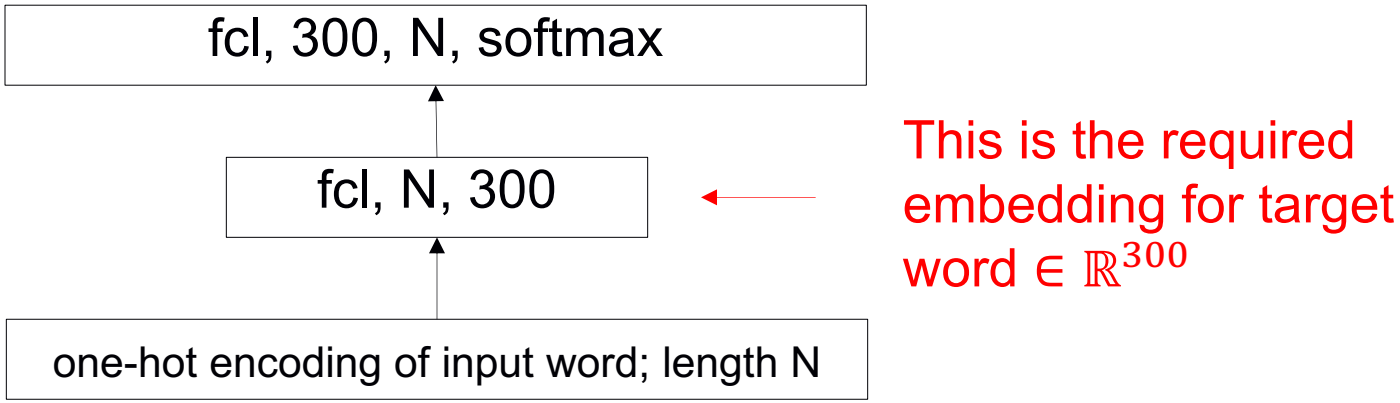
2-D projection of 60-D word-embedding

Note that similar words are close together.

Skip-gram

Word2vec is widely used. There are two principal methods – we will examine the so-called *skip-gram* method.

Given a vocabulary of N words (typically >>1,000), train a simple fully connected encoder-decoder network:



Output is the probability distribution, over all words, for a word appearing in the context window of the input word.

Train on pairs (input word, context word) from a given text corpus.

Generating training data

The rain in Spain falls mainly on the plain → (the, rain) (the, in)

The rain in Spain falls mainly on the plain → (rain, the) (rain, in) (rain, spain)

The rain in Spain falls mainly on the plain → (in, the) (in, rain) (in, spain) (in, falls)

The rain in Spain falls mainly on the plain → (spain, rain) (spain, in) (spain, falls) (spain, mainly)

Context window size = 5

Visualising the embedding space

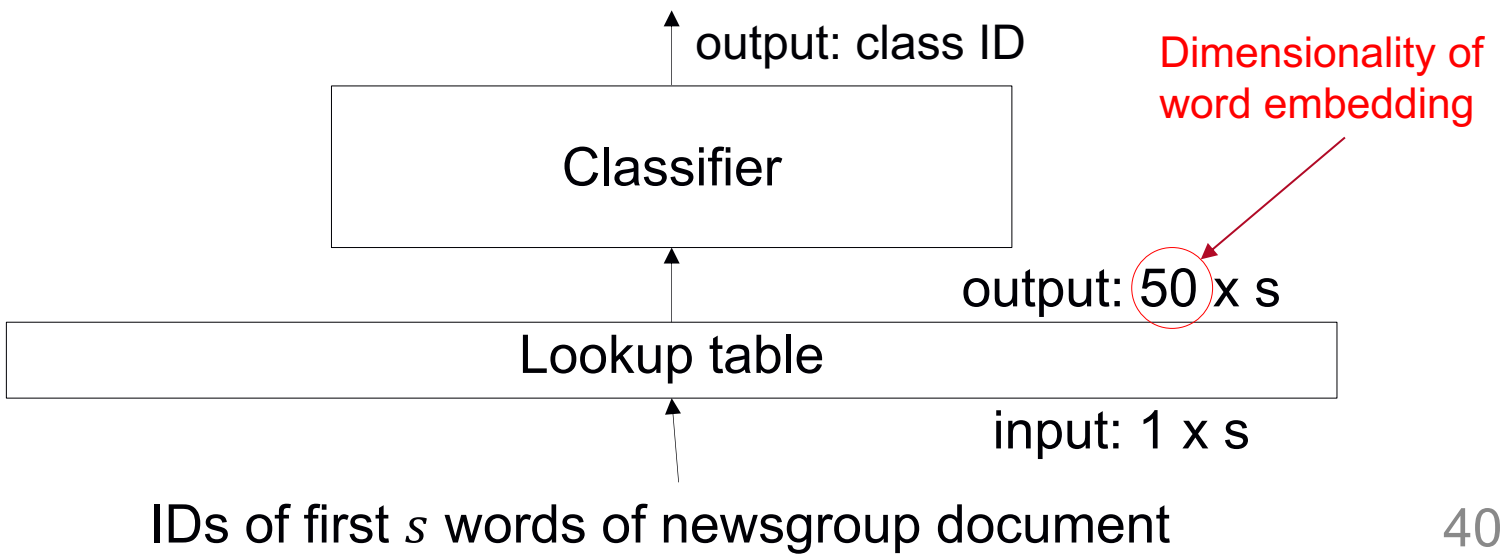
https://projector.tensorflow.org/

Example of using word embedding for text classification

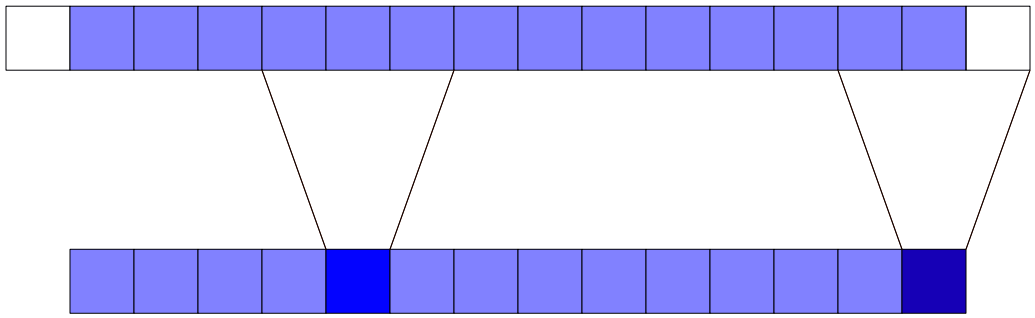
20 Newsgroups dataset
(http://qwone.com/~jason/20Newsgroups/)

20,000 newsgroup documents, 20 categories

comp.graphics comp.os.ms-windows.misc comp.sys.ibm.pc.hardware comp.sys.mac.hardware comp.windows.x	rec.autos rec.motorcycles rec.sport.baseball rec.sport.hockey	sci.crypt sci.electronics sci.med sci.space
misc.forsale	talk.politics.misc talk.politics.guns talk.politics.mideast	talk.religion.misc alt.atheism soc.religion.christian



Text classification using a CNN



We use 1-D convolution on sequences of tokens.

Sometimes referred to as *temporal convolution.*

Conneau, Alexis, Holger Schwenk, Loïc Barrault, and Yann Lecun. 2017. “Very Deep Convolutional Networks for Text Classification.” In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 1, Long Papers*, 1107–1116. Valencia, Spain: Association for Computational Linguistics. https://www.aclweb.org/anthology/E17-1104.

Text classification datasets

Data set	#Train	#Test	#Classes	Classification Task
AG's news	120k	7.6k	4	English news categorization
Sogou news	450k	60k	5	Chinese news categorization
DBPedia	560k	70k	14	Ontology classification
Yelp Review Polarity	560k	38k	2	Sentiment analysis
Yelp Review Full	650k	50k	5	Sentiment analysis
Yahoo! Answers	1 400k	60k	10	Topic classification
Amazon Review Full	3 000k	650k	5	Sentiment analysis
Amazon Review Polarity	3 600k	400k	2	Sentiment analysis

From Conneau et al. EACL 2017

Dataset example: Amazon Review Full

Sentiment analysis: predict the score (1-5) from review text.

For example, from a more recent review…

64 characters long (including spaces)

600k examples of each score, used in training

130k examples of each score used in testing

Select text lengths between 100 and 1014 characters

Input representation

Map input to vector of size s made up of IDs for 69 distinct character tokens:

abcdefghijklmnopqrstuvwxyz0123456789-,;.!?:’"/| #$%ˆ&*˜‘+=<>()[]{}

□ space

♦ used to pad input text to s characters

⊘ unknown character

```
what□can□one□say□about□a□pack□of□batteries□delivery□however□good♦♦♦ ··· ♦♦♦
```

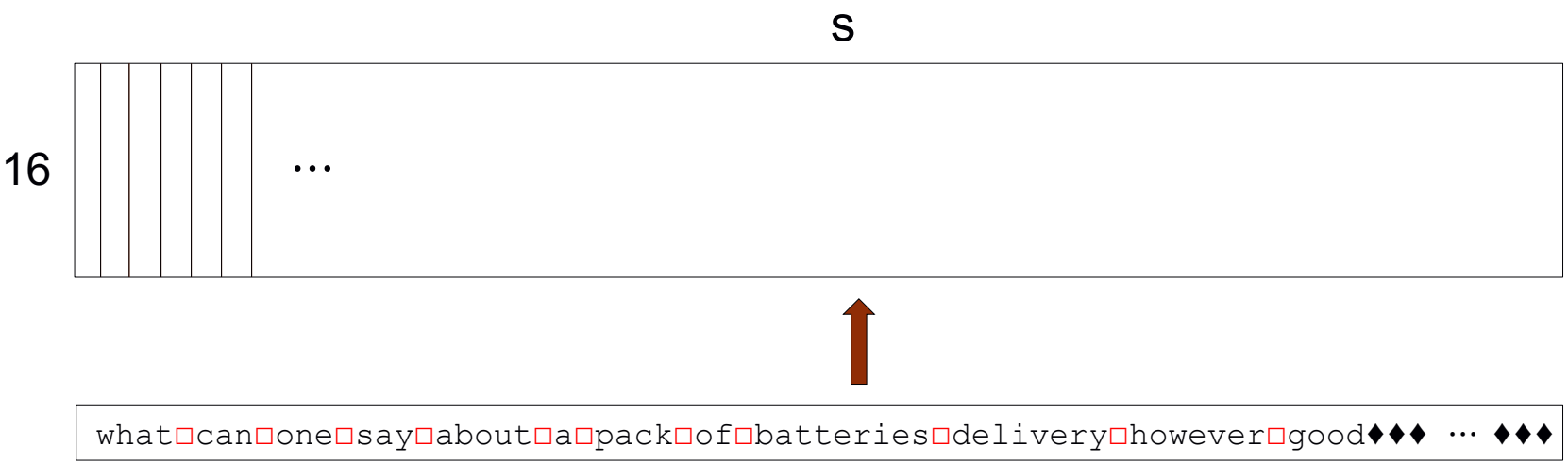
s character tokens

Generate a *character embedding*

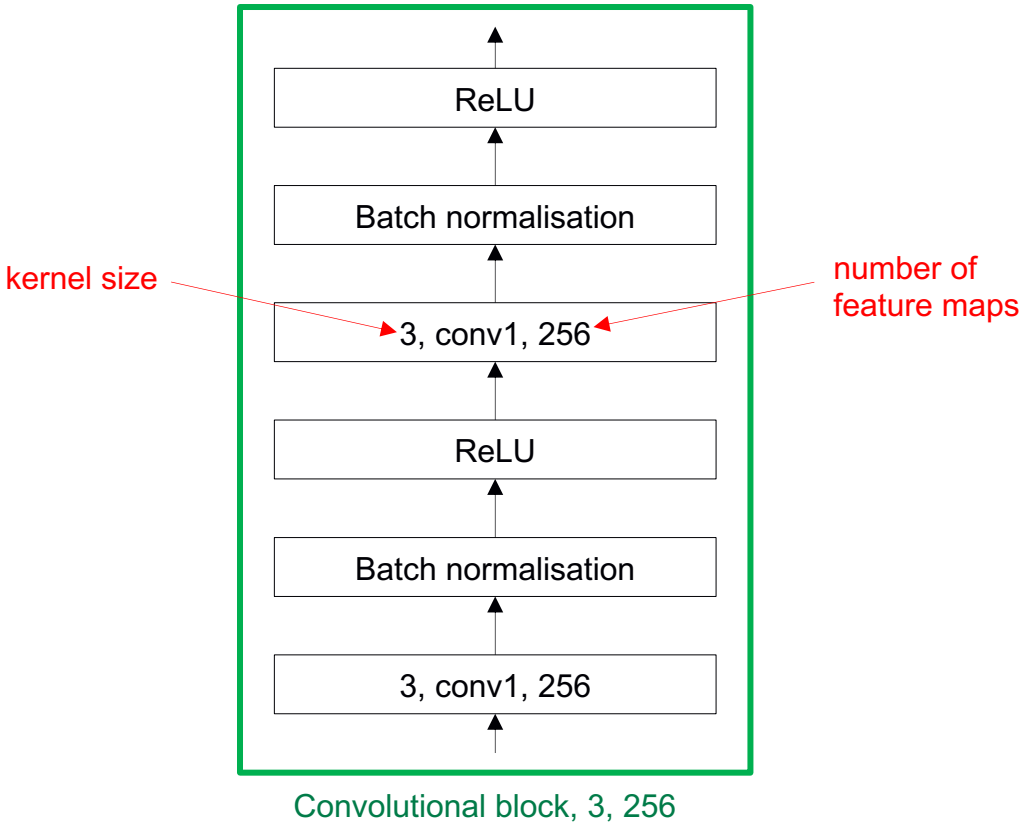
Use a 16×69 lookup table to replace each character by a corresponding vector of size 16.

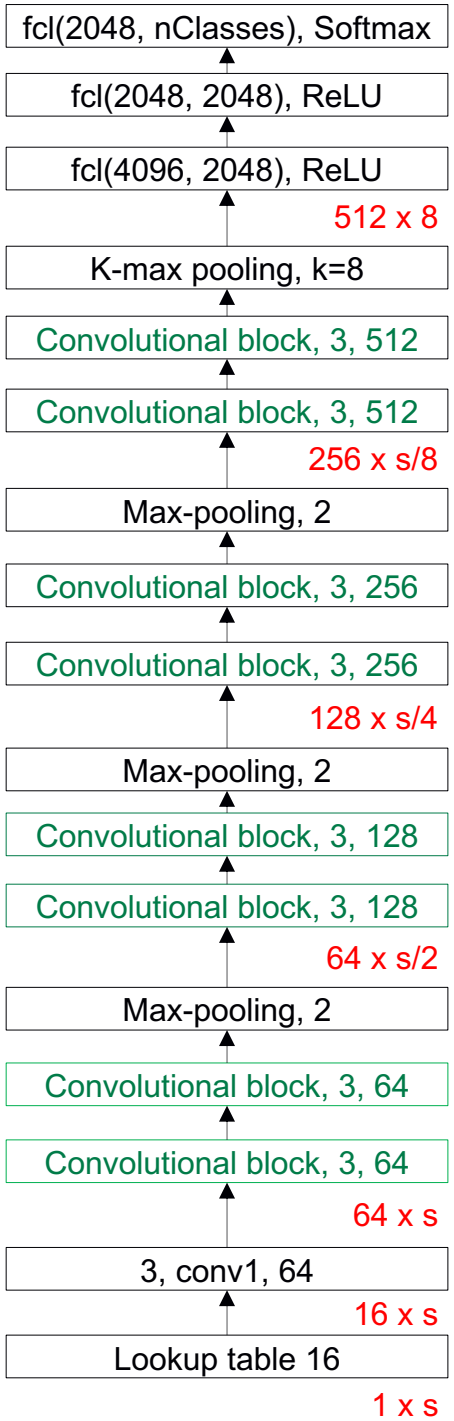
Each vector is an *embedding* of the corresponding character.

Now represent the input string as a $16 \times s$ tensor.



Network architecture





The number of feature maps doubles as their size halves – this is common in CNNs applied to images.

Token embeddings in the lookup table are learnable – no need for pre-generation.

K-max pooling

Defined in LeCun et al., 1998

Given a value k and a sequence $\mathbf{p}$ of length $\mathrm{n} \geq k$, k-max pooling selects the subsequence $\mathbf{q}$ of the k highest values of $\mathbf{p}$. The order of the values in $\mathbf{q}$ corresponds to their original order in $\mathbf{p}$.

$$\mathbf{p} = (3, 71, 9, 2, 1, 19, 27, 13, 6, 49)$$

k-max pooling on $\mathbf{p}$ for k=4

$$\mathbf{q} = (71, 19, 27, 49)$$

Thus, although the order remains the same in $\mathbf{q}$, the absolute position in $\mathbf{p}$ has been lost.